

Claude Code 完全指南：使用方式、技巧与最佳实践

Claude Code 完全指南：使用方式、技巧与最佳实践

提一嘴博主开源的以 Claude Code 为基础的 GUI 图形化桌面客户端应用 **Misaka** (
GitHub 地址: <https://github.com/knqiufan/Misaka>

项目完全开源,是个学习项目,还有比较多的地方待优化,欢迎大家来提 Issue 和 PR

另外 Claude Code 的其他相关文章:

- 关于 Skill: [从对话到协作, Skills 如何改变我们与 AI 共事的方式](#)
- 关于 SubAgent: [拆解 Claude Code SubAgent: 隔离、专业化与权限设计](#)

最近在深度使用 Claude Code,发现真的很香!即整理了一份全面的指南,从零开始逐步了解 Claude Code 的核心功能、高级特性和最佳实践。这篇博客实际上更像是一本工具书,让我自己在使用时也能随时参考其内容,更好的使用这个强大的 AI 工具。

目录

一、Claude Code 简介

1.1 什么是 Claude Code?

Claude Code(CC)是由 Anthropic 开发的**系统级 AI Agent**,它不仅是一个代码编写工具,更是一个可以通过自然语言指令完成各种电脑任务的智能助手。

核心特性:

特性	说明
全功能访问	拥有系统级权限,可执行文件操作、运行命令、管理进程等
超大上下文	支持 200K token 上下文窗口,可处理大型项目
高度可扩展	支持 MCP、Skills、Plugins、Hooks 等多种扩展方式
多代理协作	支持子代理(Subagents)并行处理复杂任务
自然交互	支持自然语言指令,无需学习复杂命令语法

1.2 Claude Code vs 传统工具

核心差异:

- 传统工具**:单一功能,需要人工操作多个工具完成复杂任务
- Claude Code**:系统级 AI Agent,通过自然语言指令即可完成全流程任务

二、安装与配置

2.1 前置准备

必需工具:

工具	用途	安装地址
Node.js	运行环境	https://nodejs.org
Git	版本控制	https://git-scm.com
API Key	模型服务	智谱GLM/月之暗面K2/阿里Qwen等

验证安装:

```
# 检查 Node.js 版本
node -v
```

```
# 检查 Git 版本
git --version
```

2.2 安装 Claude Code

全局安装(推荐):

```
npm install -g @anthropic-ai/claude-code
```

验证安装:

```
claude --version
```

2.3 配置模型

Claude Code 支持多种模型配置方式,你可以根据自己的需求选择合适的模型。

方式一:手动配置(通用方式)

手动配置适用于所有兼容 Anthropic API 的模型。配置方式如下:

Windows:

```
setx ANTHROPIC_BASE_URL "模型API地址"
setx ANTHROPIC_AUTH_TOKEN "你的API密钥"
setx ANTHROPIC_MODEL "模型名称"
```

macOS/Linux:

```
export ANTHROPIC_BASE_URL=模型API地址
export ANTHROPIC_AUTH_TOKEN=你的API密钥
export ANTHROPIC_MODEL=模型名称

# 永久配置(添加到 ~/.bashrc 或 ~/.zshrc)
echo 'export ANTHROPIC_BASE_URL=模型API地址' >> ~/.bashrc
echo 'export ANTHROPIC_AUTH_TOKEN=你的API密钥' >> ~/.bashrc
echo 'export ANTHROPIC_MODEL=模型名称' >> ~/.bashrc
source ~/.bashrc
```

常用国内模型配置示例:

配置示例(以智谱GLM为例):

```
export ANTHROPIC_BASE_URL=https://open.bigmodel.cn/api/anthropic
export ANTHROPIC_AUTH_TOKEN=your_glm_api_key
export ANTHROPIC_MODEL=GLM-4.7
```

注意: 配置环境变量后需要重启终端或运行 `source ~/.bashrc` 使配置生效。

方式二:使用自动化助手(仅适用于智谱GLM)

如果选择使用智谱GLM系列模型,还可以使用自动化配置助手:

```
npx @z_ai/coding-helper
```

按照交互式提示完成配置:

- 1. 选择界面语言
- 2. 设置 Coding 套餐
- 3. 配置 API 密钥
- 4. 选择编码工具

这种方式可以自动完成智谱GLM模型的配置,适合不想手动设置环境变量的用户。

国内模型对比

模型	提供商	代码能力	价格	优势	适用场景
glm-4.7	智谱AI	★★★★★	中等	中文理解强,有Coding套餐	中文项目为主

模型	提供商	代码能力	价格	优势	适用场景
kimi-k2-turbo-preview	月之暗面	★★★★★	较低	超长上下文,MoE架构	大型项目重构
qwen-coder-plus	阿里云	★★★★★	低	开源,性能优秀	Python/JS项目
deepseek-chat	深度求索	★★★★	极低	价格优势	预算有限的场景

2.4 启动 Claude Code

基本启动:

```
claude
```

危险模式(跳过权限确认):

```
claude --dangerously-skip-permissions
```

Headless 模式(非交互式):

```
git diff | claude -p "解释这些更改"
```

三、核心概念详解

了解了 Claude Code 的安裝配置后,深入了解一下它的一些核心概念。这些概念是充分发挥 Claude Code 能力的基础。

3.1 Skills(技能包)

什么是 Skills?

Skills 是预封装的工作流,就像游戏中的"技能包",用完即走,不占用上下文。它是别人已经编写好的、可直接使用的功能模块。

官方 Skills 库: <https://github.com/anthropics/skills> (32k+ Stars)

Skills 的类型

类型	说明	示例
User Skills	用户自定义技能,存储在本地	个人工作流自动化
Plugin Skills	插件提供的技能,随插件安装	frontend-design
Built-in Skills	Claude Code 内置技能	commit, review-pr

常用官方 Skills

```
# 前端设计技能
npx skills-installer install @anthropics/claude-code/frontend-design --client claude-code
```

```
# 文档协同技能
npx skills-installer install @anthropics/claude-code/doc-coauthoring --client claude-code
```

```
# Canvas 设计技能
npx skills-installer install @anthropics/claude-code/canvas-design --client claude-code
```

```
# PDF 处理技能
npx skills-installer install @anthropics/claude-code/pdf --client claude-code
```

```
# 算法艺术生成
npx skills-installer install @anthropics/claude-code/algorithmic-art --client claude-code
```

如何使用 Skills

查看可用 Skills:

```
claude /skills
```

调用 Skill:

```
# 在 Claude Code 对话中
使用 frontend-design skill 优化 https://example.com
```

```
使用 pdf skill 提取 report.pdf 中的表格数据
```

如何编写自己的 Skills

Skill 目录结构:

```
my-skill/
├── skill.json      # Skill 元数据
├── skill.md        # Skill 文档
├── api/            # API 定义(可选)
└── tools/          # 自定义工具(可选)
```

skill.json 示例:

```
{
  "name": "my-custom-skill",
  "description": "我的自定义技能",
  "version": "1.0.0",
  "author": "Your Name",
  "categories": ["automation"],
  "license": "MIT",
  "skill": {
    "file": "skill.md",
    "description": "这个技能用于..."
  }
}
```

skill.md 示例:

My Custom Skill

这个技能帮助用户快速完成[特定任务]。

使用场景

- 场景1:描述...
- 场景2:描述...

使用方式

用户只需要告诉你要完成什么, 这个技能就会自动:

1. 分析需求
2. 执行步骤
3. 返回结果

注意事项

- 注意事项1
- 注意事项2

安装本地 Skill:

将技能复制到 Claude Code 配置目录
cp -r my-skill ~/.claude/skills/

或使用安装命令
npx skills-installer install ./my-skill --client claude-code

3.2 Hooks(钩子)

什么是 Hooks?

Hooks 是在特定事件触发时自动执行的脚本,用于自定义工作流、拦截危险操作、自动格式化代码等。

核心价值:

graph LR
A[事件触发] --> B{Hook 拦截}
B --> C[执行自定义脚本]
C --> D[返回结果]
D --> E[继续/阻止操作]
style B fill:#f093fb
style C fill:#f5576c

Hook 事件类型

事件类型	触发时机	典型用途
user-prompt-submit	用户提交提示词前	验证、修改提示词
tool-use	工具使用前	权限检查、参数验证
after-tool-use	工具使用后	日志记录、结果处理
permission-request	权限请求时	拦截危险操作
notification	通知时	发送告警、更新状态

Hook 配置方式

方式一:通过 /hooks 命令

在 Claude Code 中
/hooks

方式二:通过配置文件

在 ~/.claude/settings.json 或项目 .claude/settings.json 中配置:

```
{
  "hooks": {
    "user-prompt-submit-hook": {
      "command": "npm run validate-prompt",
      "enabled": true
    },
    "tool-use-hook": {
      "command": "~/claude/hooks/check-permission.sh",
      "enabled": true,
      "blocking": true
    },
    "after-tool-use-hook": {
      "command": "echo 'Tool used: {{toolName}}' >> ~/.claude/hooks.log",
      "enabled": true
    }
  }
}
```

Hook 实战示例

拦截危险命令:

```
#!/bin/bash
# ~/.claude/hooks/check-dangerous.sh

# 读取工具调用信息
TOOL_NAME=$(jq -r '.toolName' <<< "$CLAUDE_HOOK_INPUT")

# 危险操作列表
DANGEROUS_TOOLS=("rm" "delete" "format" "shutdown")

if [[ "${DANGEROUS_TOOLS[@]}" =~ "${TOOL_NAME}" ]]; then
  echo "⚠️ 警告:即将执行危险操作 - $TOOL_NAME"
  echo "请确认是否继续? (yes/no)"
  read -r confirmation
  if [[ "$confirmation" != "yes" ]]; then
    exit 1 # 阻止操作
  fi
fi
```

自动格式化代码:

```
{
  "hooks": {
    "after-write-hook": {
      "command": "if [[ {{filePath}} == *.js ]]; then prettier --write {{filePath}}; fi",
      "enabled": true,
      "blocking": false
    }
  }
}
```

发送通知:

```
{
  "hooks": {
    "task-complete-hook": {
      "command": "notify-send 'Claude Code' '任务已完成'",
      "enabled": true
    }
  }
}
```

自动格式化代码(PostToolUse Hook):

来自创始人的实战经验 - 彻底消灭 CI 里的格式报错:

```
{
  "hooks": {
    "after-tool-use-hook": {
      "command": "bun run format || true",
      "enabled": true,
      "blocking": false
    }
  }
}
```

工作原理:

1. 每次 Claude 使用 Write 或 Edit 工具后自动触发
2. 运行格式化命令(这里是 bun run format)
3. || true 确保即使格式化失败也不阻塞流程
4. 虽然 Claude 已经写得很规范,但这最后 10% 的自动化处理能彻底解决格式问题

效果:

-  CI 中不再有格式报错
-  代码风格始终一致
-  无需手动运行格式化
-  Git diff 更清晰

3.3 Plugins(插件)

什么是 Plugins?

Plugins 是打包在一起的扩展集合,可以包含:

- 5 个 Skills
- 10 个斜杠命令
- 3 个 MCP 服务器配置
- 2 个 SubAgent 定义
- 若干 Hooks

Plugins vs Skills:

特性	Skills	Plugins
复杂度	简单工作流	完整功能套件
内容	单一技能	多种资源的集合
安装	独立安装	一次性安装多个资源
适用场景	单一任务	完整解决方案

Plugin 安装与使用

哪里获取已有 Plugins?

官方插件市场:

添加插件市场:

```
# 添加官方Anthropic插件市场
claude /plugin marketplace add anthropics/skills

# 添加本地插件市场
claude /plugin marketplace add ~/my-marketplace

# 浏览可用插件
claude /plugin
# 选择 "Browse Plugins" 查看完整列表
```

常用官方插件:

```
# 文档处理插件套件
claude /plugin marketplace add anthropics/skills
claude /plugin install document-skills

# 前端开发插件
claude /plugin install frontend-design

# Git工作流插件
claude /plugin install git-workflow
```

安装 Plugin

从市场安装:

```
claude plugin install <plugin-name>
```

从本地安装:

```
# 安装本地插件
claude plugin install ./my-plugin

# 或使用完整路径
claude plugin install /path/to/my-plugin
```

从GitHub安装:

```
# 直接从GitHub仓库安装
claude plugin install github:user/repo
```

查看 Plugins

查看已安装 Plugins:

```
claude /plugin
```

浏览可用插件:

```
# 在Claude Code中输入
/plugin
# 选择 "Browse Plugins"
```

卸载 Plugin:

```
claude plugin uninstall <plugin-name>
```

创建自定义 Plugin:

```
my-plugin/
├── plugin.json           # Plugin 配置
├── skills/               # Skills 目录
│   ├── skill1/
│   └── skill2/
├── commands/            # 自定义斜杠命令
│   └── my-command.md
├── mcp/                 # MCP 配置
│   └── mcp-config.json
├── agents/              # SubAgent 定义
│   └── agent1.json
└── hooks/               # Hook 脚本
    └── hook1.sh
```

plugin.json 示例:

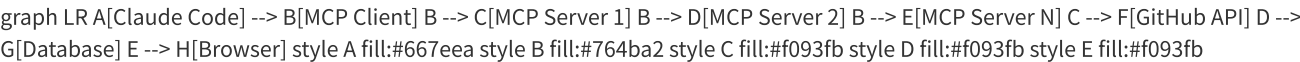
```
{
  "name": "my-plugin",
  "version": "1.0.0",
  "description": "我的自定义插件",
  "author": "Your Name",
  "skills": [
    "skills/skill1",
    "skills/skill2"
  ],
  "commands": [
    {
      "name": "/my-command",
      "description": "我的自定义命令",
      "file": "commands/my-command.md"
    }
  ],
  "mcpServers": [
    {
      "name": "my-mcp",
      "config": "mcp/mcp-config.json"
    }
  ],
  "agents": [
    {
      "name": "my-agent",
      "config": "agents/agent1.json"
    }
  ]
}
```

3.4 MCP Servers(模型上下文协议服务器)

什么是 MCP?

MCP (Model Context Protocol) 是 AI 的扩展接口标准,通过添加 MCP 服务器可以扩展 Claude Code 获取外部工具、资源、服务的能力。

核心概念:



常用 MCP 服务器

MCP Server	功能	Star 数
chrome-devtools-mcp	浏览器自动化,26个工具	18.5k
github-mcp	GitHub API 集成	10k+
postgres-mcp	PostgreSQL 数据库操作	5k+
filesystem-mcp	增强文件系统操作	3k+
web-search-mcp	网络搜索功能	2k+

MCP 安装方式

方式一:命令行安装

```
claude mcp add chrome-devtools npx chrome-devtools-mcp@latest
```

方式二:配置文件安装

编辑 ~/.claude/mcp.json:

```
{
  "mcpServers": {
    "chrome-devtools": {
      "command": "npx",
      "args": ["chrome-devtools-mcp@latest"],
      "disabled": false
    }
  }
}
```

```
    },
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": {
        "GITHUB_TOKEN": "your_github_token_here"
      }
    },
    "postgres": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-postgres"],
      "env": {
        "POSTGRES_CONNECTION_STRING": "postgres://user:password@localhost:5432/db"
      }
    }
  }
}
```

验证安装:

```
# 在 Claude Code 中
/mcp
```

```
# 或通过命令行
claude mcp list
claude mcp test chrome-devtools
```

Chrome DevTools MCP 实战

安装:

```
claude mcp add chrome-devtools npx chrome-devtools-mcp@latest
```

使用示例:

```
# 在 Claude Code 中
用Chrome浏览器打开 https://example.com,然后通过 chrome devtools mcp 完成以下任务:
1. 截取页面截图
2. 提取所有链接
3. 分析页面结构
4. 获取页面性能数据
```

26个内置工具包括:

- `chrome_navigate`: 导航到指定 URL
- `chrome_screenshot`: 截取页面截图
- `chrome_click`: 点击元素
- `chrome_fill`: 填写表单
- `chrome_select`: 选择元素
- `chrome_evaluate`: 执行 JavaScript
- 等等...

3.5 Subagents(子代理)

什么是 Subagents?

Subagents 是可以并行处理任务的独立 AI 代理,每个子代理拥有独立的 200K 上下文窗口,可以分配不同任务以提高效率。

核心理念: 把常用 workflow 看作自动化运行的"子智能体",就像圣诞老人分派任务给精灵一样,每个子智能体专注于特定领域。

核心优势:

graph TB
 A[主代理 Claude Code] --> B[子代理1: 代码审查]
 A --> C[子代理2: 测试编写]
 A --> D[子代理3: 文档生成]
 A --> E[子代理4: 性能优化]
 B --> F[并行处理]
 C --> F
 D --> F
 E --> F
 F --> G[结果汇总]

Subagent 配置

方式一:通过 /agents 命令

```
claude /agents
```

方式二:配置文件

在 `~/claude/agents.json` 或项目 `.claude/agents.json` 中配置:

```
{
  "agents": [
    {
      "code-reviewer": {
        "description": "专门负责代码审查的子代理",
        "model": "claude-opus-4-5",
        "instructions": "你是一个专业的代码审查专家,专注于检查代码质量、安全漏洞和性能问题。",
        "tools": ["read", "search", "git"],
        "permissions": {
          "allowWrite": false
        }
      },
      "test-writer": {
        "description": "专门负责编写测试的子代理",
```



```
    "model": "claude-sonnet-4-5",
    "instructions": "你是一个测试工程师,专注于编写全面的单元测试和集成测试。",
    "tools": ["read", "write", "bash"]
  },
  "doc-generator": {
    "description": "专门负责生成文档的子代理",
    "model": "claude-sonnet-4-5",
    "instructions": "你是一个技术文档专家,专注于生成清晰、准确的技术文档。",
    "tools": ["read", "write"]
  }
}
```

Subagent 使用示例

场景:完成一个功能开发

主任务
我需要完成用户认证功能,请帮我:

- 使用 code-reviewer agent 审查现有认证代码
- 使用 test-writer agent 编写测试用例
- 使用 doc-generator agent 更新 API 文档

这三个任务并行执行

Claude Code 会自动:

- 创建三个独立的子代理
- 分配各自的上下文(200K × 3)
- 并行执行任务
- 汇总结果返回

实战子代理案例

来自 Claude Code 创始人 Boris Cherny 的实际使用案例:

code-simplifier:

.claude/agents/code-simplifier.md

你是一个代码精简专家。在 Claude 完成工作后,你的任务是:

- 分析代码的复杂度和可读性
- 识别可以简化的逻辑
- 提供优化建议但保持功能不变
- 优先考虑性能和可维护性

verify-app:

.claude/agents/verify-app.md

你是一个端到端测试专家。你的任务是验证应用功能:

- 运行完整的测试套件
- 检查所有关键路径
- 验证边界情况
- 确保用户体验“感觉对劲”
- 如果发现问题,提供详细的修复步骤

使用方式:

在 Claude Code 中
使用 code-simplifier agent 优化刚才写的代码

使用 verify-app agent 验证应用是否正常工作

3.6 CLAUDE.md(项目记忆文件)

什么是 CLAUDE.md?

CLAUDE.md 是 Claude Code 的"项目记忆文件",记录项目结构、构建命令、代码规范、架构决策等信息,让 Claude Code 快速理解项目上下文。

CLAUDE.md 的作用

作用	说明
 项目知识库	记录项目架构、技术栈、依赖关系
 快速启动	自动读取,无需重复解释项目背景
 团队协作	共享项目规范,确保团队理解一致
 持续迭代	随项目演进自动更新

CLAUDE.md 最佳位置

graph TD
 A[项目根目录_CLAUDE.md] --> B[src_.claude_CLAUDE.md]
 B --> C[.claude_rules_auth.md]
 C --> D[home_.claude_CLAUDE.md]
 D --> E[全局项目配置]
 E --> F[模块级配置]
 F --> G[特定规则]
 G --> H[用户级配置]

style A fill:#667eea style B fill:#764ba2 style C fill:#f093fb style D fill:#f5576c style E fill:#667eea style F fill:#764ba2 style G fill:#f093fb style H fill:#f5576c

优先级: 特定规则 > 模块配置 > 项目配置 > 用户配置

CLAUDE.md 示例

完整示例:

```
# 项目名称: E-Commerce Platform

## 项目概述
这是一个基于 Node.js + React 的电商平台,支持商品管理、订单处理、支付集成等功能。

## 技术栈
- **前端:** React 18, TypeScript, Tailwind CSS, Redux Toolkit
- **后端:** Node.js 20, Express, TypeScript
- **数据库:** PostgreSQL 15, Redis 7
- **认证:** JWT, OAuth 2.0
- **测试:** Jest, Playwright
- **部署:** Docker, Kubernetes

## 项目结构
\`\`\`
src/
├── frontend/           # React 前端
│   ├── components/    # 可复用组件
│   ├── pages/         # 页面组件
│   ├── store/         # Redux store
│   └── utils/         # 工具函数
├── backend/           # Node.js 后端
│   ├── controllers/   # 控制器
│   ├── services/      # 业务逻辑
│   ├── models/        # 数据模型
│   └── routes/        # API 路由
├── shared/            # 共享代码
└── types/             # TypeScript 类型定义
\`\`\`

## 常用命令

### 开发环境
\`\`\`bash
# 安装依赖
npm install

# 启动前端开发服务器
npm run dev:frontend

# 启动后端开发服务器
npm run dev:backend

# 同时启动前后端
npm run dev
\`\`\`

### 构建与部署
\`\`\`bash
# 构建前端
npm run build:frontend

# 构建后端
npm run build:backend

# 构建所有
npm run build

# Docker 构建
docker-compose build
docker-compose up
\`\`\`

### 测试
\`\`\`bash
# 运行所有测试
npm test

# 前端单元测试
npm run test:frontend

# 后端单元测试
npm run test:backend

# E2E 测试
npm run test:e2e

# 测试覆盖率
npm run test:coverage
\`\`\`

### 代码质量
\`\`\`bash
# 代码格式化
npm run format

# 代码检查
npm run lint
```

```
# 类型检查
npm run type-check
\\`\\`

## 代码规范

### 命名规范
- **文件名:** kebab-case (user-profile.ts)
- **组件名:** PascalCase (UserProfile)
- **函数/变量:** camelCase (getUserProfile)
- **常量:** UPPER_SNAKE_CASE (API_BASE_URL)
- **类型/接口:** PascalCase (UserProfile)

### Git 提交规范
遵循 Conventional Commits:
- \`feat: 新功能\`
- \`fix: 修复 bug\`
- \`docs: 文档更新\`
- \`style: 代码格式调整\`
- \`refactor: 代码重构\`
- \`test: 测试相关\`
- \`chore: 构建/工具链更新\`

### 代码审查清单
- [ ] 代码符合项目命名规范
- [ ] 添加了必要的注释
- [ ] 更新了相关文档
- [ ] 编写了/更新了测试
- [ ] 通过了所有测试
- [ ] 通过了 lint 检查
- [ ] 没有引入安全漏洞

## 架构决策

### ADR-001: 选择 TypeScript 而非 JavaScript
**日期:** 2024-01-15
**状态:** 已接受
**理由:**
- 类型安全减少运行时错误
- 更好的 IDE 支持
- 代码可维护性更高

### ADR-002: 采用微服务架构
**日期:** 2024-02-20
**状态:** 已接受
**理由:**
- 便于团队并行开发
- 独立部署和扩展
- 技术栈灵活性

## 环境变量

### 必需变量
\\`\\`\\`bash
DATABASE_URL=postgresql://...
REDIS_URL=redis://...
JWT_SECRET=your-secret-key
API_BASE_URL=https://api.example.com
\\`\\`\\`

### 可选变量
\\`\\`\\`bash
LOG_LEVEL=info
NODE_ENV=development
PORT=3000
\\`\\`\\`

## 常见问题

### Q: 如何添加新的 API 端点?
A:
1. 在 \\`backend/routes/\\` 创建路由文件
2. 在 \\`backend/controllers/\\` 创建控制器
3. 在 \\`backend/services/\\` 实现业务逻辑
4. 添加测试用例
5. 更新 API 文档

### Q: 如何调试前端状态管理问题?
A:
1. 使用 Redux DevTools 浏览器扩展
2. 在 \\`src/frontend/store/\\` 添加日志
3. 检查 action 和 reducer 逻辑

## 重要注意事项

1. **安全:** 永远不要在代码中硬编码密钥或敏感信息
2. **性能:** 大数据查询必须使用分页
3. **测试:** 所有新功能必须包含测试
4. **文档:** API 变更必须更新文档
5. **兼容性:** 确保向后兼容,使用版本控制

## 相关资源
- [项目 Wiki](https://wiki.example.com)
- [API 文档](https://docs.example.com)
- [设计规范](https://design.example.com)
```

生成 CLAUDE.md 的方式

方式一:使用 /init 命令

```
# 在项目根目录
claude /init
```

Claude Code 会自动扫描项目并生成初始 CLAUDE.md,包含:

- 构建和测试命令
- 目录结构说明
- 代码规范和架构决策
- 技术栈信息

方式二:手动创建

```
# 创建基础文件
touch CLAUDE.md

# 让 Claude Code 帮助生成
claude "请根据当前项目结构生成 CLAUDE.md 文件"
```

方式三:模块化规则

```
# 在 .claude/rules/ 目录创建多个规则文件
.claude/rules/
├── auth.md           # 认证相关规则
├── database.md       # 数据库相关规则
├── api.md            # API 设计规范
└── testing.md        # 测试规范
```

方式四:Memory Updates - 动态更新记忆

```
# 直接告诉 Claude 更新知识
Update CLAUDE.md: always use bun instead of npm
Update CLAUDE.md: 不要使用 enum,改用 string union

# Claude 会自动把新知识写入记忆文件,无需手动编辑
```

CLAUDE.md 的 AI 进化机制

来自 Claude Code 创始人团队的实战经验:

核心理念: 让 Claude 在 Code Review 中自我迭代,越用越聪明

实战流程:

1. 在 PR 中发现问题

```
# 在 GitHub PR 评论中
@claude 这里的代码使用了 enum,但我们项目规范要求使用 string union,请修复
```

2. 让 Claude 记住教训

```
# 在 PR 中直接告诉 Claude
@claude 请把这次的教训写入 CLAUDE.md:不要使用 enum,改用 string union
```

3. Claude 自动更新

```
# Claude 会在 CLAUDE.md 中添加
## 代码规范更新 (2026-01-03)

### Enum vs String Union
- ❌ 不要使用 enum
- ✅ 改用 string union
- 理由:更好的类型推断和 Tree-shaking
```

4. 团队共同维护

```
# 将 CLAUDE.md 签入 Git
git add CLAUDE.md
git commit -m "docs: 更新 CLAUDE.md 规范"

# 整个团队共享这份“行为准则”
```

价值体现:

- 🧠 **集体智慧:** 每个团队成员的反馈都让 AI 更聪明
- 🔄 **持续进化:** Claude 不会重复犯同样的错误
- 📖 **知识沉淀:** 项目规范自动文档化
- 🤝 **团队协作:** 统一的 AI 助手理解团队偏好

实际案例:

```
# CLAUDE.md - 实际演进示例

## 初始版本 (第1周)
- 使用 TypeScript
- 测试覆盖率 > 80%

## 第1次更新 (第2周 - PR反馈)
+ 永远使用 bun 而不是 npm
+ 理由:启动速度快 10 倍

## 第2次更新 (第3周 - PR反馈)
+ 不要使用 enum,改用 string union
+ 理由:更好的类型推断
```

四、高级功能

实际上CC还有一些高级功能，这些功能将帮助更高效地完成复杂任务。

4.1 Plan 模式(规划模式)

什么是 Plan 模式？

Plan 模式是一种"先规划、后执行"的工作模式,Claude 会先分析项目架构、依赖关系并起草实现方案,确认后才开始编写代码。

Anthropic 开发者关系负责人 Ado Kukic 有 90% 的时间都在使用这个模式。

核心价值: 在这个模式下,Claude 会阅读代码、分析架构、**起草计划,但绝不修改代码**。直到你批准计划,它才会动手。你是架构师,它是执行者。

核心价值:

graph LR A[接收任务] --> B[分析项目] B --> C[探索代码库] C --> D[设计方案] D --> E{用户确认?} E -->|是| F[执行实现] E -->|否| G[调整方案] G --> D F --> H[完成任务] style B fill:#667eea style C fill:#764ba2 style D fill:#f093fb style E fill:#f5576c style F fill:#4facfe

进入 Plan 模式

快捷键:

按两次 Shift+Tab
Shift+Tab, Shift+Tab

命令方式:

/plan

Plan 模式工作流程

1. 探索阶段

- 读取相关文件
- 分析代码结构
- 理解依赖关系

2. 规划阶段

- 设计实现方案
- 列出具体的步骤
- 识别潜在风险

3. 确认阶段

- 展示完整计划
- 等待用户反馈
- 根据反馈调整

4. 执行阶段

- 按计划实施
- 实时反馈进度
- 处理异常情况

Plan 模式最佳实践

适合场景:

- ☒ 复杂功能开发(多文件、多步骤)
- ☒ 架构重构
- ☒ 性能优化
- ☒ 代码迁移
- ☒ 不熟悉的项目

不适合场景:

- ❌ 简单 bug 修复
- ❌ 单行代码修改
- ❌ 文档查询
- ❌ 快速原型验证

使用技巧:

```
# 启用 Plan 模式
Shift+Tab × 2

# 明确任务需求
请帮我实现用户认证功能, 包括:
1. 用户注册
2. 用户登录
3. JWT token 验证
4. 密码加密存储

# Claude 会先探索并规划:
# Plan: 实现用户认证功能
#
# 1. 分析现有代码结构
# 2. 设计认证流程
# 3. 创建数据模型
# 4. 实现 API 端点
# 5. 添加中间件
# 6. 编写测试
#
# 确认后开始执行? (yes/no)

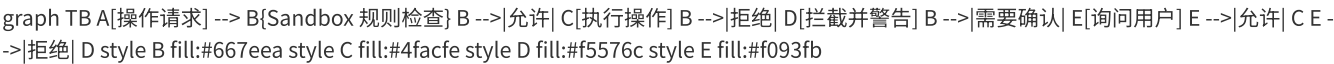
# 确认后开始实施
yes
```

4.2 Sandbox 模式(沙箱模式)

什么是 Sandbox 模式?

Sandbox 模式通过定义允许的操作范围,拦截危险操作,提高安全性。

核心机制:



配置 Sandbox 模式

方式一:通过 /permissions 命令

```
claude /permissions
```

方式二:配置文件

编辑 ~/.claude/settings.json:

```
{
  "permissions": {
    "allow": {
      "bash": [
        "npm install",
        "npm test",
        "npm run build",
        "git *",
        "node -v",
        "npm -v"
      ],
      "write": [
        "src/**/*",
        "tests/**/*",
        "*.md"
      ],
      "read": [
        "**/*"
      ]
    },
    "deny": {
      "bash": [
        "rm -rf *",
        "format *",
        "shutdown",
        "reboot"
      ],
      "write": [
        "node_modules/**/*",
        ".git/**/*",
        "/etc/*",
        "/usr/*"
      ]
    }
  }
}
```

Sandbox 最佳实践

最小权限原则:

```
{
  "permissions": {
    "allow": {
      "bash": ["npm test", "npm run build"],
      "write": ["src/**/*", "tests/**/*"]
    }
  }
}
```

特定项目配置:

```
// .claude/settings.json (项目级)
{
  "permissions": {
    "allow": {
      "bash": [
        "npm run dev:*",
        "npm run test:*",
        "npm run build:*"
      ],
      "write": ["src/**/*", "tests/**/*", "docs/**/*"]
    },
    "deny": {
      "write": ["dist/**/*", "build/**/*"]
    }
  }
}
```

4.3 Headless 模式(无头模式)

什么是 Headless 模式？

Headless 模式是非交互式运行方式,输出结果直接流向标准输出,可集成到 Shell 脚本或 CI/CD 流程中。

使用场景:

场景	示例
 CI/CD 集成	自动化代码审查
 脚本自动化	批量处理任务
 快速分析	不需要交互的代码分析
 报告生成	自动生成文档

Headless 模式使用

基本用法:

```
# 从管道输入
git diff | claude -p "解释这些更改"

# 从文件输入
cat file.txt | claude -p "总结这个文件"

# 直接指定
claude -p "检查代码质量" < src/main.js
```

CI/CD 集成示例:

```
# .github/workflows/claude-review.yml
name: Claude Code Review

on:
  pull_request:
    types: [opened, synchronize]

jobs:
  review:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Setup Node.js
        uses: actions/setup-node@v3
        with:
          node-version: '20'

      - name: Install Claude Code
        run: npm install -g @anthropic-ai/claude-code

      - name: Run Code Review
        run: |
          git diff origin/main...HEAD | claude -p "审查这个 PR 的代码变更" > review.txt

      - name: Comment PR
        uses: actions/github-script@v6
        with:
          script: |
            const fs = require('fs');
```

```
const review = fs.readFileSync('review.txt', 'utf8');
github.rest.issues.createComment({
  issue_number: context.issue.number,
  owner: context.repo.owner,
  repo: context.repo.repo,
  body: review
});
```

脚本自动化示例:

```
#!/bin/bash
# batch-process.sh

for file in src/**/*.js; do
  echo "Processing $file..."

  # 使用 Claude Code 分析代码
  claude -p "检查文件 $file 的代码质量,包括:
1. 潜在的 bug
2. 性能问题
3. 安全漏洞
4. 代码规范违规" < "$file" > "reports/${basename $file}.md"

  echo "✓ Completed $file"
done

echo "All files processed!"
```

4.4 Vim 模式

启用 Vim 模式

/vim

Vim 常用快捷键

快捷键	功能
h	左移光标
j	下移光标
k	上移光标
l	右移光标
ciw	修改单词
dd	删除行
yy	复制行
p	粘贴
u	撤销
Ctrl+r	重做

4.5 Slash Commands(自定义命令)

什么是 Slash Commands?

Slash Commands 是将高频工作流封装成可复用的斜杠命令,将复杂的"人机对话"变成简单的"命令行指令"。

来自 Claude Code 创始人的实战经验:为了避免重复输入相同的 Prompt,他将高频工作流封装成了 Slash Commands。

创建自定义命令

方式一:通过 /commands 命令

```
claude /commands
# 选择 "Create new command"
```

方式二:手动创建

在 .claude/commands/ 目录创建 Markdown 文件:

```
# .claude/commands/
└── commit-push-pr.md
```



```
└─── daily-standup.md
└─── deploy-staging.md
```

示例1:自动提交并创建 PR

```
# .claude/commands/commit-push-pr.md
```

你是一个发布助手。请执行以下步骤：

- 检查 Git 状态
!git status
- 运行测试套件
!npm test
- 如果测试通过：
 - 添加所有更改
 - 生成符合 Conventional Commits 的提交消息
 - 推送到远程
 - 创建 Pull Request
- 如果测试失败：
 - 分析失败原因
 - 提供修复建议

使用:

```
# 直接输入命令
/commit-push-pr
```

Claude 会自动执行整个流程

示例2:生成每日站会日报

```
# .claude/commands/daily-standup.md
```

请生成今日站会报告, 包括:

- 昨天完成了什么
!git log --since="yesterday" --oneline
- 今天计划做什么
(读取 CLAUDE.md 中的当前任务)
- 遇到的阻碍
(读取最近的 TODO 注释)

请以清晰的 Markdown 格式输出。

使用:

```
/daily-standup
```

实战案例

完整的部署工作流:

```
# .claude/commands/deploy-production.md
```

你是一个部署专家。请按以下步骤执行生产环境部署：

- ```
前置检查
1. 确认当前分支
!git branch --show-current

2. 检查是否有未提交的更改
!git status

3. 运行完整测试套件
!npm run test:all

构建阶段
4. 构建生产版本
!npm run build

5. 运行构建产物测试
!npm run test:build

部署阶段
6. 打包部署
!npm run deploy:staging

7. 验证部署
!curl https://staging.example.com/health

8. 运行冒烟测试
!npm run test:smoke

生产部署
9. 合并到主分支
!git checkout main
!git merge production

10. 推送到远程
!git push origin main

11. 监控部署
 (等待 CI/CD 完成)

12. 验证生产环境
!curl https://api.example.com/health

回滚准备
```

如果部署失败,立即执行:  
!git revert HEAD  
!git push origin main

## 部署后检查

13. 检查错误日志  
(使用 Sentry MCP 查询最新错误)

14. 发送团队通知  
(使用 Slack MCP 发送部署成功通知)

请每完成一步都报告进度,遇到问题立即暂停并询问。

价值:

-  一键执行复杂流程
-  减少人为错误
-  团队共享最佳实践
-  可以签入 Git 版本控制

## 4.6 Extended Thinking(扩展思考模式)

ultrathink 深度思考模式

当你需要设计复杂的缓存层或重构架构时,在提示词中加上 ultrathink。

# 标准模式  
设计一个 Redis 缓存层

# ultrathink 模式  
ultrathink: 设计一个高可用的 Redis 缓存层,考虑:  
- 缓存穿透、缓存击穿、缓存雪崩  
- 分布式锁  
- 缓存更新策略  
- 降级方案

效果:

- Claude 会分配高达 32k 的 Token 进行内部推理
- 虽然反应慢一点,但逻辑准确率大幅提升
- 适合架构设计、复杂重构等关键任务

Extended Thinking API

通过 API 调用时,开启 Extended Thinking 可以看到 Claude 的逐步推理过程(Thinking Blocks)。

价值:

- 透明化思考过程
- 便于调试复杂逻辑
- 理解 Claude 的决策路径
- 教学和学习工具

---

## 五、实用技巧与快捷操作

了解了 Claude Code 的高级功能后,分享一些实用的技巧和快捷操作,可以更高效地使用这个工具。

### 5.1 基础操作技巧

项目初始化(/init)

# 自动生成 CLAUDE.md  
/init

# 或手动指定  
claude /init “这是一个 Node.js + React 项目”

快速引用上下文(@提及)

# 引用单个文件  
@src/auth.ts

# 引用整个目录  
@src/components/

# 引用多个文件  
@src/auth.ts @src/user.ts @src/database.ts

# 引用 MCP 服务器  
@mcp:github

```
模糊匹配
@auth # 自动匹配 auth.ts, auth.controller.ts 等
```

### 即时执行 Bash 命令(!前缀)

```
查看状态
!git status

运行测试
!npm test

查看进程
!ps aux | grep node

组合使用
!git diff && echo "=== Changes Summary ==="
```

### 回退操作(双击 ESC)

ESC ESC

#### 选项:

- 仅回退对话
- 仅回退代码
- 同时回退对话和代码

**注意:** 已执行的 Bash 命令无法回退

## 5.2 效率提升技巧

### 反向搜索历史提示词(Ctrl+R)

```
Ctrl+R # 开始搜索
Ctrl+R # 循环匹配项
Enter # 运行
Tab # 编辑后运行
```

### 提示词暂存(Ctrl+S)

```
Ctrl+S # 暂存当前提示词
... 处理其他事务 ...
恢复暂存内容继续工作
```

### 类似 Git stash,但用于提示词

### 智能提示补全(Prompt Suggestions)

Claude 甚至能预测你接下来想问什么:

```
当你看到灰色的建议文字时
Tab # 接受建议并编辑
Enter # 直接运行建议
```

#### 价值:

- 减少打字量
- 发现隐藏功能
- 学习最佳实践

## 会话管理高级技巧

### Continue & Resume - 无缝续关

```
终端意外关闭?电脑没电了?
claude --continue # 瞬间恢复上一次对话
claude --resume # 显示历史会话列表供选择
```

# 上下文完美保留,工作流永不丢失

### Named Sessions - 会话命名

```
像管理 Git 分支一样管理会话
/rename api-migration # 给当前会话命名
/resume api-migration # 按名称恢复会话
```

# 用途:为不同任务创建独立会话

```
/rename feature-auth
/rename bugfix-login
/resume feature-auth
```

### Claude Code Remote - 跨设备传送

```
在网页版 claude.ai/code 上开始任务
回家后用终端继续开发
```

```
claude --teleport session_id # 把云端会话“拉”到本地
```

# 利用“传送 (Teleport)”功能  
# 在终端和网页间同步上下文  
# 甚至早上起床和白天休息时,用手机版 Claude App 查看进度

## 自定义状态栏(/statusline)

/statusline

### 可显示信息:

- Git 分支
- 当前模型
- Token 用量
- 上下文占比
- 任务进度

### 配置示例:

```
{
 "statusline": {
 "segments": [
 "git.branch",
 "model.name",
 "context.usage",
 "token.cost"
],
 "refreshInterval": 1000
 }
}
```

## 可视化上下文(/context)

/context

### 显示:

- 当前 token 使用情况
- 上下文占用百分比
- 各文件占用大小
- 建议优化方向

## 使用统计与监控

### 查看使用习惯:

/stats # 查看你的使用习惯、最爱用的模型、连续使用天数等  
/usage # 查看当前的费率限制和使用进度

### 价值:

- 了解自己的使用模式
- 监控费用消耗
- 晒 Claude Code 统计(现在比晒 GitHub 提交记录更流行)

## 5.3 文件夹管理技巧

### 工作空间隔离

#### 最佳实践:

# 为每个任务创建独立文件夹  
project-1/  
project-2/  
task-a/  
task-b/

#### 快速启动:

# Windows: 在地址栏输入 cmd  
# macOS: 在右键菜单选择“服务 > 新建终端位于文件夹”  
  
# 然后启动 Claude Code  
claude

### 拖拽文件

### 支持操作:

- 拖拽单个文件
- 拖拽整个文件夹
- 拖拽多个文件

**使用场景:**  
# 拖拽文件后直接描述任务  
[拖入 auth.ts]  
这个认证模块有安全问题, 请帮我审查并修复

5.4 粘贴技巧

文本粘贴

| 操作         | 方式                                      |
|------------|-----------------------------------------|
| 在 CC 中粘贴文本 | 鼠标右键 → 粘贴                               |
| 在普通命令行粘贴   | Ctrl+V                                  |
| 在 CC 中粘贴图片 | 复制图片 → Alt+V (Windows) / Ctrl+V (macOS) |

- 注意:
- CC 中不能用 Ctrl+V 粘贴文本
  - CC 中不能用 Ctrl+A 全选
  - 图片必须在打开状态复制,预览状态无法粘贴

5.5 常用斜杠命令速查

| 命令       | 功能         | 使用频率  |
|----------|------------|-------|
| /clear   | 清空对话历史     | ★★★★★ |
| /compact | 清空对话但保留摘要  | ★★★★★ |
| /context | 可视化上下文使用   | ★★★★★ |
| /model   | 切换模型       | ★★★★  |
| /cost    | 显示费用统计     | ★★★★  |
| /export  | 导出对话       | ★★★★  |
| /add-dir | 添加工作目录     | ★★★★  |
| /status  | 查看系统状态     | ★★★   |
| /mcp     | 管理 MCP 服务器 | ★★★   |
| /skills  | 列出可用技能     | ★★★   |
| /hooks   | 管理钩子       | ★★    |
| /agents  | 管理子代理      | ★★    |
| /vim     | 切换 Vim 模式  | ★★    |
| /theme   | 更换主题       | ★     |
| /doctor  | 诊断环境       | ★★★★  |

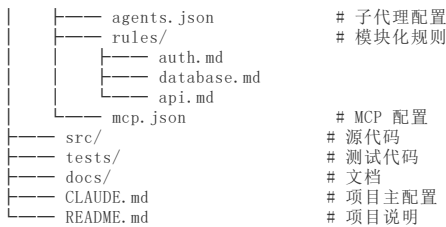
六、最佳实践

继续。提供并建立一些最佳实践以提升团队协作和开发效率。

6.1 项目组织最佳实践

目录结构规范

```
project/
├── .claude/ # Claude Code 配置
└── settings.json # 项目级设置
```



## CLAUDE.md 层级配置

```
graph TD
 A[用户级_claude_CLAUDE_md] --> B[项目级_CLAUDE_md]
 B --> C[模块级_src_claude_CLAUDE_md]
 C --> D[特定规则_claude_rules]
 D --> E[合并规则]
 E --> F[最终配置]
```

## 6.2 工作流最佳实践

## 探索-规划-编码-提交工作流

这是一个完整的开发工作流程,需要配合特定的Skills和MCP使用。

### 前置准备 - 安装必要工具:

```
1. 安装代码分析MCP(可选,用于深度分析)
claude mcp add github npx -y @modelcontextprotocol/server-github

2. 安装commit skill(用于生成规范的commit消息)
npx skills-installer install @anthropics/claude-code/commit --client claude-code

3. 安装Chrome DevTools MCP(用于浏览器测试,可选)
claude mcp add chrome-devtools npx chrome-devtools-mcp@latest
```

### 详细工作流程:

## 第1步:探索阶段 - 理解项目

```
启动Claude Code
claude

探索项目架构
请帮我分析这个项目的架构, 重点关注:
1. 整体项目结构
2. 认证模块的实现
3. 依赖关系
4. 技术栈

使用@引用特定目录
@src/

如果有GitHub仓库, 使用GitHub MCP
请使用github MCP查看最近的commit历史和issue
```

## 第2步:规划阶段 - 使用Plan模式

```
进入Plan模式(按两次Shift+Tab)
Shift+Tab, Shift+Tab

或使用命令
/plan

明确任务需求
我需要添加 OAuth2.0 认证支持, 请规划实施方案, 包括:
1. 需要创建哪些新文件
2. 需要修改哪些现有文件
3. 需要安装哪些依赖
4. 实现步骤顺序
5. 潜在的风险点

Claude会生成详细计划, 等待你的确认
Plan: 添加OAuth2.0认证支持
#
1. 分析现有认证代码
2. 设计OAuth2.0流程
3. 创建OAuth控制器
4. 实现token生成和验证
5. 添加测试用例
6. 更新文档
#
预计需要修改的文件:
- src/auth/oauth.controller.ts (新建)
- src/auth/auth.service.ts (修改)
- src/config/oauth.config.ts (新建)
#
确认后开始执行? (yes/no)

确认后开始实施
yes
```

### 第3步:编码阶段 - 实施计划

```
让Claude按照计划实施
按照上面的计划开始实施

如果遇到问题,可以要求暂停
```

暂停,让我先检查一下这一步

# 继续实施  
继续下一步

# 运行测试  
!npm test

# 修复失败的测试  
测试失败了,请帮我修复

#### 第4步:提交阶段 - 使用commit skill

# 使用commit skill生成规范的commit消息  
使用commit skill为这些更改创建一个规范的commit消息

# Claude会分析变更并生成commit  
# commit skill会:  
# 1. 分析git diff  
# 2. 生成符合Conventional Commits的消息  
# 3. 自动创建commit

# 或手动指导commit  
请帮我创建git commit,包含这些更改  
commit消息格式: feat(auth): 添加OAuth2.0认证支持

### 测试驱动开发(TDD)工作流

#### 前置准备 - 安装测试相关工具:

# 1. 安装测试编写skill(可选)  
npx skills-installer install @anthropics/claude-code/test-writer --client claude-code

# 2. 或者配置test-writer子代理(后面详细说明)

#### TDD工作流程:

# 第1步:先写测试  
# 使用test-writer skill或agent  
使用test-writer skill为用户登录功能编写测试,包括:  
1. 正常登录场景  
2. 错误密码场景  
3. 用户不存在场景  
4. Token验证场景

@src/auth/login.controller.ts

# 第2步:运行测试(预期失败)  
!npm test

# 第3步:实现最小可行代码  
测试失败了,请实现登录功能使测试通过,只写能通过的代码

# 第4步:重构代码  
代码通过了但不够优雅,请重构以提高可读性,但保持测试通过

# 第5步:重复循环  
继续添加新功能的测试...

### 代码审查工作流

#### 前置准备 - 创建和配置代码审查子代理:

##### 方式1:使用 /agents 命令创建

# 在Claude Code中  
claude  
/agents

# 选择 "Create new agent"  
# 按提示配置:  
- Name: code-reviewer  
- Description: 专门负责代码审查的子代理  
- Instructions: 你是一个代码审查专家,专注于检查代码质量、安全漏洞和性能问题  
- Tools: read, search, git  
- Permissions: 禁止写入(allowWrite: false)

##### 方式2:手动配置文件

# 创建agents配置文件  
mkdir -p ~/.claude  
cat > ~/.claude/agents.json << 'EOF'  
{  
 "agents": {  
 "code-reviewer": {  
 "description": "专门负责代码审查的子代理",  
 "model": "claude-sonnet-4-5",  
 "instructions": "你是一个专业的代码审查专家。专注于:\n1. 代码质量(Clean Code原则)\n2. 安全漏洞(OWASP Top 10)\n3. 性能问题(算法复杂度、资源使用)",  
 "tools": ["read", "search", "git"],  
 "permissions": {  
 "allowWrite": false  
 }  
 },  
 "security-reviewer": {  
 "description": "专注于安全问题的子代理",  
 "model": "claude-sonnet-4-5",  
 "instructions": "你是一个安全专家,专注于识别:\n1. SQL注入、XSS、CSRF等漏洞\n2. 敏感信息泄露\n3. 权限控制问题\n4. 依赖安全问题",  
 "tools": ["read", "search"],  
 "permissions": {  
 "allowWrite": false  
 }  
 }  
 }  
}

```
 },
 "performance-reviewer": {
 "description": "专注于性能问题的子代理",
 "model": "claude-sonnet-4-5",
 "instructions": "你是一个性能优化专家,专注于:\n1. 算法效率\n2. 内存使用\n3. 数据库查询优化\n4. 缓存策略",
 "tools": ["read", "search"],
 "permissions": {
 "allowWrite": false
 }
 }
 }
}
EOF

验证配置
claude
/agents
应该看到刚创建的三个agent
```

### 使用代码审查 workflow:

```
第1步:使用子代理并行审查
代码质量审查
使用code-reviewer agent审查以下文件的代码质量:
- 是否符合Clean Code原则
- 是否有代码异味
- 是否易于维护

@src/auth.ts @src/user.ts @src/database.ts

安全审查(并行)
使用security-reviewer agent审查这些文件的安全问题:
- SQL注入风险
- XSS漏洞
- 敏感信息泄露

@src/auth.ts @src/user.ts @src/database.ts

性能审查(并行)
使用performance-reviewer agent审查这些文件的性能问题:
- 算法复杂度
- 数据库查询效率
- 内存使用

@src/auth.ts @src/user.ts @src/database.ts

第2步:生成综合审查报告
请综合以上三个agent的审查结果,生成一份详细报告,包括:
- 发现的问题列表
- 每个问题的严重程度(Critical/High/Medium/Low)
- 具体的修复建议
- 修复优先级排序

第3步:逐个修复问题
根据审查报告,请帮我修复Critical和High级别的问题:
1. 先修复安全问题
2. 再修复性能问题
3. 最后修复代码质量问题

第4步:重新审查
修复完成后,使用code-reviewer agent重新审查修复后的代码
```

## 6.3 团队协作最佳实践

### 共享 CLAUDE.md

场景:团队项目

#### 实施步骤:

1. 在项目根目录创建 CLAUDE.md
2. 团队共同维护
3. 定期更新
4. 版本控制

#### 示例:

```
团队项目规范

团队约定
- 代码审查至少 1 人 approve
- 所有 PR 必须通过 CI
- 文档与代码同步更新

分工信息
- @alice: 前端负责人
- @bob: 后端负责人
- @charlie: DevOps

沟通渠道
- Slack: #team-channel
- JIRA: 项目看板
```

### GitHub 集成



## 安装 GitHub App:

/install-github-app

## 使用 PR 评论:

```
在 PR 中
@. claude 请审查这个 PR 的代码变更

自动添加问题到 CLAUDE.md
/pr-comments
```

## 6.4 性能优化最佳实践

### 极致的并行处理(Parallelism)

来自 Claude Code 创始人 Boris Cherny 的核心理念:不要让 AI 等你,也不要让你等 AI

#### 1. 终端里的"多线程"操作

```
同时运行 5 个 Claude 实例
将终端标签页标记为 1-5

配置系统通知(System Notifications)
当某个 Claude 完成任务需要输入时,直接弹窗提醒

在 ~/.claude/settings.json 中配置
{
 "notifications": {
 "enabled": true,
 "sound": true,
 "position": "top-right"
 }
}
```

就好比你雇了 5 个初级程序员同时干活,你只需要负责 Review 和分配任务,效率直接翻倍。

#### 2. 本地与云端的"混合双打"

```
在本地终端运行 5 个实例
同时在 claude.ai/code 网页端运行 5-10 个会话

利用 & 命令将本地会话"甩"给网页端后台运行
& 将这个任务转到网页端继续

在网页版开始新任务
然后在终端用 claude --teleport session_id 拉回本地
```

#### 3. 跨设备同步工作流

```
早上在办公室:终端 + 网页版
中午休息:手机版 Claude App 查看进度
晚上回家:claude --teleport 继续开发

工作流永不中断
```

### 效果对比:

| 方式         | 并发度  | 效率提升   | 适用场景 |
|------------|------|--------|------|
| 单实例        | 1x   | 基准     | 简单任务 |
| 5个终端       | 5x   | 400%   | 本地开发 |
| 5终端 + 10网页 | 15x  | 1400%  | 大型项目 |
| 全设备协作      | 20x+ | 1900%+ | 复杂重构 |

### 上下文管理

#### 策略:

```
长对话定期 compact
/compact "保留以下要点: ..."

新任务先 clear
/clear

监控上下文使用
/context

定期 export 备份
/export conversation-2026-01-06.json
```

### Token 优化

| 技巧           | 说明             | 节省效果   |
|--------------|----------------|--------|
| 使用 @提及       | 只加载需要的文件       | 30-50% |
| 定期 /compact  | 清理历史保留摘要       | 40-60% |
| 避免重复加载       | 检查 /context 输出 | 20-30% |
| 使用 Subagents | 并行处理减少串行上下文    | 50-70% |

模型选择策略

Claude Code 创始人的独家见解: 只用最强的模型:Opus 4.5 + Thinking

很多人为了省钱或追求速度会用 Haiku 或 Sonnet,但 Boris Cherny 只用 Opus 4.5,并全程开启 Thinking(思考)模式。

理由:

- 虽然 Opus 4.5 更贵、更慢,但它不需要你频繁纠正(Steer less)
- 工具使用能力更强,一次就能做对
- 从"任务完成总耗时"来看,聪明的大模型反而比"快但笨"的小模型更快
- 一次做对,永远比反复修改更省时间

土豪专属了属于是。

国外模型选择策略:

graph LR A[简单任务] --> B[Haiku 4.5] C[中等复杂度] --> D[Sonnet 4.5] E[高复杂度] --> F[Opus 4.5 + Thinking] B --> G[成本优化] D --> H[平衡性能] F --> I[质量优先] style A fill:#4facfe style C fill:#667eea style E fill:#f5576c style B fill:#4facfe style D fill:#667eea style F fill:#f5576c

国外模型使用建议:

- **快速查询/格式化:** Haiku 4.5 - 最快最便宜
- **日常开发/代码编写:** Sonnet 4.5 - 性价比平衡
- **架构设计/复杂重构:** Opus 4.5 + Thinking - 最高质量(创始人首选)

国内模型选择策略:

graph LR A[简单任务] --> B[DeepSeek-Coder] C[中文项目] --> D[GLM-4.7] E[大型项目] --> F[Kimi K2] G[Python/JS] --> H[Qwen-Coder-Plus] B --> I[极低成本] D --> J[中文优化] F --> K[超长上下文] H --> L[开源优秀] style A fill:#00d2ff style C fill:#667eea style E fill:#f093fb style G fill:#4facfe style B fill:#00d2ff style D fill:#667eea style F fill:#f093fb style H fill:#4facfe

国内模型使用建议:

| 任务类型      | 推荐模型            | 理由               | 价格(相对) |
|-----------|-----------------|------------------|--------|
| 简单查询      | DeepSeek-Coder  | 极低成本,够用          | ★      |
| 中文项目      | GLM-4.7         | 中文理解最强,有Coding套餐 | ★★★    |
| 大型重构      | Kimi K2         | 超长上下文(2M+),MoE架构 | ★★     |
| Python/JS | Qwen-Coder-Plus | 开源,Python/JS性能优秀 | ★★     |
| 预算有限      | DeepSeek-Coder  | 价格优势明显           | ★      |
| 团队协作      | GLM-4.7         | 有团队套餐,管理方便       | ★★★    |

模型切换策略:

```
方式1:使用 /model 命令快速切换
claude
/model
选择对应模型

方式2:临时切换(当前会话)
export ANTHROPIC_MODEL=kimi-k2
claude

方式3:针对不同任务使用不同配置
创建alias
alias cc-glm=' ANTHROPIC_MODEL=GLM-4.7 claude'
alias cc-kimi=' ANTHROPIC_MODEL=kimi-k2 claude'
alias cc-qwen=' ANTHROPIC_MODEL=qwen-coder-plus claude'

使用
cc-glm # 使用智谱GLM
cc-kimi # 使用Kimi
cc-qwen # 使用通义千问
```

成本优化建议:

graph TB
 A[任务开始] --> B{任务类型?}
 B -->|简单查询| C[DeepSeek-Coder]
 B -->|中文开发| D[GLM-4.7]
 B -->|大型重构| E[Kimi K2]
 B -->|Python项目| F[Qwen-Coder-Plus]
 B -->|高质量要求| G[Sonnet 4.5]
 C --> H[成本: \$]
 D --> I[成本: \$\$]
 E --> J[成本: \$\$\$]
 F --> K[成本: \$\$\$]
 G --> L[成本: \$\$\$\$]
 style A fill:#667eea
 style C fill:#00d2ff
 style D fill:#f093fb
 style E fill:#f5576c
 style F fill:#4facfe
 style G fill:#764ba2

混合使用策略:

```
日常开发用国产模型 (便宜)
export ANTHROPIC_MODEL=GLM-4.7

遇到复杂任务时临时切换到高质量模型
/model
选择 Opus 4.5

完成后切换回国产模型
/model
选择 GLM-4.7
```

6.5 安全最佳实践

Sandbox 配置

```
{
 "permissions": {
 "allow": {
 "bash": ["npm run *", "git *"],
 "write": ["src/**/*", "tests/**/*"],
 "read": ["**/*.ts", "**/*.js", "**/*.json"]
 },
 "deny": {
 "bash": ["rm -rf *", "format *", "shutdown"],
 "write": ["node_modules/**/*", ".git/**/*", "/etc/*", "/usr/*"]
 }
 }
}
```

Hooks 安全检查

```
#!/bin/bash
~/.claude/hooks/security-check.sh

检查敏感操作
if [["$TOOL_NAME" == "Bash"]]; then
 COMMAND=$(jq -r '.command' <<< "$CLAUDE_HOOK_INPUT")

 # 危险命令黑名单
 DANGEROUS_CMDS=("rm -rf" "format" "shutdown" "reboot" "chmod 000")

 for cmd in "${DANGEROUS_CMDS[@]}; do
 if [["$COMMAND" == *"$cmd"*]]; then
 echo "🚫 拦截危险命令: $COMMAND"
 exit 1
 fi
 done
fi
```

敏感信息保护

```
CLAUDE.md 安全规范

禁止事项
- ❌ 在代码中硬编码密钥
- ❌ 在 CLAUDE.md 中记录密码
- ❌ 在 Git 提交中包含凭证

推荐做法
- ✅ 使用环境变量
- ✅ 使用 .env 文件 (不提交)
- ✅ 使用密钥管理服务 (AWS Secrets Manager 等)
- ✅ .gitignore 排除敏感文件
```

6.6 验证闭环(Feedback Loop)

来自 Claude Code 创始人的最重要的一条技巧：永远给 Claude 一种验证自己工作的方法

核心原理

如果 Claude 能看到自己代码的运行结果(报错信息、测试通过与否),它的代码质量会提升 2-3 倍。

实践方法

1. 简单的任务:运行 Bash 命令验证

```
编写代码后立即验证
请实现一个用户登录功能,然后:
1. 运行测试
!npm test

根据测试结果修复问题
```

```
重新验证
!npm test

直到所有测试通过
```

## 2. 复杂的 UI:使用浏览器自动化

```
使用 Claude Chrome 扩展
请修复这个 Bug 并验证:
1. 用 Chrome 打开 http://localhost:3000/login
2. 输入测试账号和密码
3. 点击登录按钮
4. 检查是否正确跳转到首页
5. 查看控制台是否有报错
6. 直到体验“感觉对劲”为止
```

## 3. 端到端验证流程

```
创建完整的验证循环
使用 verify-app agent 验证以下功能:
1. 用户注册流程
2. 登录和登出
3. 权限验证
4. 数据持久化
```

```
运行完整测试套件:
!npm run test:e2e
```

如果发现问题, 请:

- 分析根因
- 修复代码
- 重新验证
- 确认问题已解决

### 效果对比

| 验证方式        | 代码质量  | 所需时间 | 返工率 |
|-------------|-------|------|-----|
| 无验证         | ☆☆    | 基准   | 40% |
| 人工验证        | ☆☆☆   | +50% | 20% |
| Claude 自动验证 | ☆☆☆☆☆ | +20% | 5%  |

### 为什么验证循环如此重要？

#### 人类开发流程:

- 写代码
- 手动测试
- 发现 bug
- 修复 bug
- 重复测试

#### Claude + 验证循环流程:

- Claude 写代码
- Claude 自己测试
- Claude 看到错误
- Claude 立即修复
- Claude 再次验证
- 循环直到完美

#### 关键差异:

- Claude 能立即看到错误信息
- Claude 不需要等待人工反馈
- Claude 可以在几秒内完成多个修复循环
- 最终代码质量远超人工编写

### 最佳实践建议

✅ **推荐做法:**

```
总是让 Claude 验证自己的工作
请实现功能 X, 然后:
1. 运行测试验证
2. 检查代码格式
3. 验证边界情况
4. 确认无回归问题
```

❌ 避免做法:

```
不要让 Claude“盲写”
请实现功能 X
(没有验证步骤, Claude 不知道代码是否正确)
```

## 七、实战案例

简单几个案例看看如何运用到实际工作中。

### 7.1 案例1:批量文件重命名

需求:将文件夹中所有文件名改为规范的英文名称

实现:

```
拖拽文件夹到 Claude Code
```

请将文件夹中的所有文件名改成规范的英文名称, 只改名字, 不改序号

```
Claude Code 自动:
1. 读取文件夹内容
2. 分析文件名
3. 批量重命名
4. 报告结果
```

### 7.2 案例2:自动化数据抓取

需求:抓取公众号多页文章数据并导出 Excel

实现:

```
使用 Chrome DevTools MCP
```

用Chrome浏览器打开这个链接:[公众号链接]

然后通过 chrome devtools mcp 完成:

- 获取第1、2、3页每篇文章的详细数据
- 包括标题、阅读量、点赞量、发布时间等
- 保存到 Excel 表格中

```
Claude Code 自动:
1. 启动 Chrome
2. 导航到页面
3. 翻页抓取数据
4. 生成 Excel 报告
```

效果对比:

| 方式                | 耗时   | 准确率 |
|-------------------|------|-----|
| 人工                | 30分钟 | 90% |
| Claude Code + MCP | 5分钟  | 99% |

### 7.3 案例3:代码质量审查

需求:审查 PR 的代码变更

实现:

```
使用 code-reviewer subagent
```

使用 code-reviewer agent 审查这个 PR:

- 代码质量
- 安全漏洞
- 性能问题
- 最佳实践

```
@src/ @tests/
```

```
Claude Code 自动:
1. 读取所有相关文件
2. 应用审查标准
3. 生成详细报告
4. 提供修复建议
```

### 7.4 案例4:自动化测试生成

需求:为新功能编写测试

实现:

```
使用 test-writer subagent
```

使用 test-writer agent 为用户认证功能编写测试:

- 单元测试
- 集成测试
- E2E 测试
- 边界情况测试

@src/auth.ts @src/auth.controller.ts

# Claude Code 自动:  
# 1. 分析代码逻辑  
# 2. 识别测试场景  
# 3. 编写测试用例  
# 4. 运行测试验证

## 7.5 案例5:CI/CD 集成

**需求:**在 PR 创建时自动审查代码

**实现:**

```
.github/workflows/claude-review.yml

name: Claude Auto Review

on:
 pull_request:
 types: [opened, synchronize]

jobs:
 review:
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v3

 - name: Install Claude Code
 run: npm install -g @anthropic-ai/claude-code

 - name: Configure Claude
 run: |
 echo "配置模型..."
 export ANTHROPIC_MODEL="claude-sonnet-4-5"

 - name: Run Review
 run: |
 git diff origin/main...HEAD > diff.txt
 claude -p "审查这个 PR 的代码变更, 重点关注:
 1. 安全漏洞
 2. 性能问题
 3. 代码规范
 请提供详细审查意见" < diff.txt > review.txt

 - name: Comment on PR
 uses: actions/github-script@v6
 with:
 script: |
 const fs = require('fs');
 const review = fs.readFileSync('review.txt', 'utf8');
 github.rest.issues.createComment({
 issue_number: context.issue.number,
 owner: context.repo.owner,
 repo: context.repo.repo,
 body: `## 🤖 Claude Code 自动审查\n\n${review}`
 });
```

## 八、常见问题与解决方案

在使用 Claude Code 的过程中,你可能会遇到一些常见问题。本章节收集了最常见的问题及其解决方案,帮助你快速排除故障。

### 8.1 安装问题

#### Q1: npm install 失败

**问题:**

```
npm ERR! code EACCES
npm ERR! syscall access
```

**解决方案:**

```
方式1: 使用 sudo (macOS/Linux)
sudo npm install -g @anthropic-ai/claude-code
```

```
方式2: 修改 npm 全局目录
mkdir ~/.npm-global
npm config set prefix '~/.npm-global'
export PATH=~/.npm-global/bin:$PATH
npm install -g @anthropic-ai/claude-code
```

```
方式3: 使用 nvm (推荐)
nvm install --lts
npm install -g @anthropic-ai/claude-code
```

#### Q2: claude 命令未找到

#### 问题:

bash: claude: command not found

#### 解决方案:

```
检查 npm 全局路径
npm config get prefix

添加到 PATH
export PATH=$(npm config get prefix)/bin:$PATH

永久配置
echo 'export PATH=$(npm config get prefix)/bin:$PATH' >> ~/.bashrc
source ~/.bashrc
```

## 8.2 配置问题

### Q3: 模型配置不生效

问题:配置后仍显示默认模型

#### 解决方案:

```
1. 验证环境变量
echo $ANTHROPIC_BASE_URL
echo $ANTHROPIC_AUTH_TOKEN
echo $ANTHROPIC_MODEL

2. 重新启动终端
配置环境变量后必须重启终端才能生效

3. 使用 claude /status 检查
claude
/status

4. 手动设置(临时)
claude
/model
选择第4个(自定义模型)
```

### Q4: MCP 服务器无法连接

#### 问题:

Error: MCP server 'chrome-devtools' failed to start

#### 解决方案:

```
1. 检查 MCP 状态
claude
/mcp

2. 测试连接
claude mcp test chrome-devtools

3. 重新安装
claude mcp remove chrome-devtools
claude mcp add chrome-devtools npx chrome-devtools-mcp@latest

4. 检查网络
npx chrome-devtools-mcp@latest --version

5. 查看日志
claude /doctor
```

## 8.3 使用问题

### Q5: 上下文超出限制

#### 问题:

Context exceeded: Request too large

#### 解决方案:

```
1. 查看上下文使用
/context

2. 使用 compact 保留摘要
/compact "保留项目背景和当前任务"

3. 使用 clear 清空历史
/clear

4. 减少文件引用
不要用 @src/ 引用整个目录
改用 @src/file1.ts @src/file2.ts 精确引用

5. 使用 Subagents 分担上下文
将大任务拆分给多个子代理
```

### Q6: 代码生成质量差

**问题:**生成的代码不符合项目规范

**解决方案:**

```
1. 完善 CLAUDE.md
在 CLAUDE.md 中明确代码规范

2. 使用 Plan 模式
Shift+Tab × 2
让 Claude 先理解项目再编写代码

3. 提供示例
这是符合规范的代码示例:
@src/good-example.ts
请参照这个风格编写新功能

4. 明确要求
请编写符合以下规范的代码:
- 使用 TypeScript 严格模式
- 遵循项目命名规范
- 包含完整的错误处理
- 添加必要的注释
```

## 8.4 性能问题

### Q7: 响应速度慢

**问题:**Claude Code 响应延迟高

**解决方案:**

```
1. 检查网络
ping api.anthropic.com

2. 切换更快的模型
/model
选择 Haiku 4.5(最快) 或国产模型

3. 优化提示词
避免冗长的提示词, 直接说明需求

4. 减少上下文
/context
如果接近 100%, 使用 /compact 清理

5. 使用本地模型(如果有)
export ANTHROPIC_BASE_URL=http://localhost:8000
```

### Q8: Token 消耗过快

**问题:**费用增长过快

**解决方案:**

```
1. 监控费用
/cost

2. 定期 compact
/compact "保留关键信息"

3. 新任务先 clear
/clear

4. 使用更便宜的模型
Haiku < Sonnet < Opus
国产模型(更便宜)

5. 优化工作流
避免反复修改, 一次性明确需求
```

## 8.5 调试技巧

### 启用调试模式

```
设置环境变量
export CLAUDE_DEBUG=true
export CLAUDE_LOG_LEVEL=debug

启动 Claude Code
claude

查看日志
tail -f ~/.claude/logs/claude.log
```

### 使用 /doctor 诊断

```
claude
/doctor
```

**检查项:**



-  安装版本
-  配置文件
-  API 连接
-  MCP 服务器
-  权限设置
-  环境变量

## 九、高级集成与扩展

Claude Code 的真正强大之处在于它的可扩展性。通过 LSP 集成和 Agent SDK，甚至可以构建自己的智能开发环境。

### 9.1 LSP 集成

#### 什么是 LSP?

**LSP (Language Server Protocol)** 是语言服务器协议，通过集成 LSP，Claude Code 现在的代码理解能力达到了 **IDE 级别**。

#### LSP 的强大能力

- # Claude Code 现在可以：
- 看到实时报错和警告
- 跳转到定义
- 查看类型信息
- 理解符号引用
- 分析代码结构

#### 效果对比:

| 能力   | 无 LSP | 有 LSP |
|------|-------|-------|
| 错误检测 | ☆☆☆   | ☆☆☆☆☆ |
| 类型推断 | ☆☆☆   | ☆☆☆☆☆ |
| 代码导航 | ☆☆    | ☆☆☆☆☆ |
| 重构建议 | ☆☆☆   | ☆☆☆☆☆ |

#### 实战案例:

- # Claude Code 可以像 IDE 一样理解代码
- @src/auth.ts
- 这个函数的返回类型是什么？
- # (Claude 会使用 LSP 查看准确的类型定义)
- 这个函数在哪里被调用了？
- # (Claude 会使用 LSP 查找所有引用)
- 这里有个类型错误, 怎么修复？
- # (Claude 会看到实时报错并提供建议)

#### 配置 LSP

Claude Code 会自动检测项目中的 LSP 服务器:

- # TypeScript 项目
- # 自动使用 tsserver
- # Python 项目
- # 自动使用 pylsp
- # Go 项目
- # 自动使用 gopls

#### 手动配置 LSP:

```
// ~/.claude/settings.json
{
 "lsp": {
 "typescript": {
 "command": "typescript-language-server",
 "args": ["--stdio"]
 },
 "python": {
 "command": "pylsp",
 "args": ["--stdio"]
 }
 }
}
```

## 9.2 Claude Agent SDK

### 什么是 Agent SDK?

**Claude Agent SDK** 将 Claude Code 的核心能力(Agent Loop、工具管理、上下文管理)作为一个 SDK 开放了。可以用几十行代码构建一个像 Claude Code 一样强大的自定义智能体。

#### 核心能力

```
import { Agent, Tool, Context } from '@anthropic-ai/agent-sdk';

// 创建自定义 Agent
const myAgent = new Agent({
 model: 'claude-sonnet-4-5',
 tools: [
 new Tool({
 name: 'readFile',
 execute: async (path: string) => {
 return fs.readFileSync(path, 'utf-8');
 }
 }),
 new Tool({
 name: 'writeFile',
 execute: async (path: string, content: string) => {
 fs.writeFileSync(path, content);
 }
 })
],
 context: new Context({
 maxSize: 200_000, // 200K tokens
 compression: 'auto'
 })
});

// 运行 Agent
const result = await myAgent.run('创建一个用户认证系统');
```

### 实战案例

#### 案例1:自动化测试生成器

```
import { Agent, Tool } from '@anthropic-ai/agent-sdk';

class TestGenerator extends Agent {
 constructor() {
 super({
 model: 'claude-sonnet-4-5',
 instructions: '你是一个测试专家, 专门为代码编写全面的测试用例',
 tools: [
 new Tool({
 name: 'readSource',
 description: '读取源代码文件',
 execute: async (path: string) => {
 return fs.readFileSync(path, 'utf-8');
 }
 }),
 new Tool({
 name: 'writeTest',
 description: '编写测试文件',
 execute: async (path: string, content: string) => {
 fs.writeFileSync(path, content);
 }
 }),
 new Tool({
 name: 'runTests',
 description: '运行测试',
 execute: async () => {
 return execSync('npm test').toString();
 }
 })
]
 });
 }
}

// 使用
const testGen = new TestGenerator();
await testGen.run(`
 为 src/auth.ts 编写测试, 包括:
 1. 正常登录场景
 2. 错误密码场景
 3. 用户不存在场景
 4. Token 验证场景

 然后运行测试确保所有测试通过
`);
```

#### 案例2:代码审查机器人

```
import { Agent, Tool } from '@anthropic-ai/agent-sdk';

class CodeReviewer extends Agent {
 constructor() {
 super({
 model: 'claude-opus-4-5',
 instructions: `
 你是一个代码审查专家。审查代码时关注:
 1. 安全漏洞(OWASP Top 10)
 `
 });
 }
}
```

2. 性能问题
3. 代码规范
4. 最佳实践

输出格式:

- 严重程度(Critical/High/Medium/Low)
- 问题描述
- 修复建议
- 代码示例

```
,
tools: [
 new Tool({
 name: 'readFile',
 execute: async (path: string) => {
 return fs.readFileSync(path, 'utf-8');
 }
 }),
 new Tool({
 name: 'gitDiff',
 execute: async () => {
 return execSync('git diff').toString();
 }
 })
]
});
}
}

// 使用
const reviewer = new CodeReviewer();
const review = await reviewer.run('审查当前的代码变更');

console.log(review);
/*
输出:
🚨 Critical: SQL 注入风险

位置: src/auth.ts:45
问题: 直接拼接 SQL 查询, 存在注入风险
建议: 使用参数化查询

代码示例:
\`\`\`typescript
// ❌ 不安全
const query = `SELECT * FROM users WHERE id = ${userId}`;

// ✅ 安全
const query = 'SELECT * FROM users WHERE id = ?';
await db.query(query, [userId]);
\`\`\`
*/
```

### 案例3:自动化部署助手

```
import { Agent, Tool } from '@anthropic-ai/agent-sdk';

class DeploymentBot extends Agent {
 constructor() {
 super({
 model: 'claude-sonnet-4-5',
 instructions: `
 你是一个部署专家。部署流程:
 1. 运行测试
 2. 构建项目
 3. 部署到 staging
 4. 验证部署
 5. 部署到生产
 6. 监控错误
 如果任何步骤失败,立即回滚
 `,
 tools: [
 new Tool({
 name: 'runTests',
 execute: async () => {
 return execSync('npm test').toString();
 }
 }),
 new Tool({
 name: 'build',
 execute: async () => {
 return execSync('npm run build').toString();
 }
 }),
 new Tool({
 name: 'deploy',
 execute: async (env: 'staging' | 'production') => {
 return execSync(`npm run deploy:${env}`).toString();
 }
 }),
 new Tool({
 name: 'verify',
 execute: async (url: string) => {
 return fetch(`${url}/health`).then(r => r.json());
 }
 }),
 new Tool({
 name: 'rollback',
 execute: async () => {
 return execSync('npm run rollback').toString();
 }
 }),
 new Tool({
 name: 'notify',
 execute: async (message: string) => {

```

```
 // 发送 Slack 通知
 await slackClient.chat.postMessage({
 channel: '#deployments',
 text: message
 });
 }
 })
 }
});
}

// 使用
const deployBot = new DeploymentBot();
await deployBot.run('部署最新版本到生产环境');
```

## Agent SDK 的核心优势

### 1. 完整的 Agent Loop

// 自动处理:

- 思考(Thinking)
- 工具调用(Tool Use)
- 结果观察(Observation)
- 迭代优化(Iteration)

### 2. 强大的上下文管理

```
const context = new Context({
 maxSize: 200_000,
 compression: 'auto', // 自动压缩历史对话
 priority: 'recent' // 优先保留最近的内容
});
```

### 3. 灵活的工具系统

// 支持各种类型的工具

- 文件操作
- API 调用
- 数据库查询
- 网络请求
- 自定义脚本

### 4. 可观测性

```
const agent = new Agent({
 onThink: (thought) => console.log('思考:', thought),
 onToolUse: (tool, args) => console.log('调用工具:', tool),
 onError: (error) => console.error('错误:', error),
 onComplete: (result) => console.log('完成:', result)
});
```

## 使用场景

| 场景       | 价值          |
|----------|-------------|
| CI/CD 集成 | 自动化代码审查和测试  |
| 自动化运维    | 智能部署和监控     |
| 文档生成     | 自动生成 API 文档 |
| 代码迁移     | 批量重构和升级     |
| 教学工具     | 编程辅导和练习     |

## 附录

### A. 快速参考卡

#### 核心命令速查

|                    |                  |
|--------------------|------------------|
| # 基础操作             |                  |
| claude             | # 启动 Claude Code |
| claude -p "prompt" | # Headless 模式    |
| claude --version   | # 查看版本           |
| # 斜杠命令             |                  |
| /clear             | # 清空对话           |
| /compact           | # 压缩对话           |
| /context           | # 查看上下文          |
| /cost              | # 查看费用           |
| /model             | # 切换模型           |
| /mcp               | # 管理 MCP         |
| /skills            | # 查看 Skills      |
| /hooks             | # 管理 Hooks       |
| /agents            | # 管理子代理          |
| /status            | # 系统状态           |
| /doctor            | # 诊断环境           |

# 快捷键  
Ctrl+R  
Ctrl+S  
Ctrl+C  
Shift+Tab × 2  
ESC ESC  
Alt+V

# 搜索历史  
# 暂存提示词  
# 中止操作  
# Plan 模式  
# 回退操作  
# 粘贴图片

# 文件操作  
@file.ts  
@src/  
!command

# 引用文件  
# 引用目录  
# 执行 Bash

B. 配置文件清单

| 配置文件          | 位置                      | 作用         |
|---------------|-------------------------|------------|
| CLAUDE.md     | 项目根目录                   | 项目配置       |
| settings.json | ~/.claude/ 或项目/.claude/ | 全局/项目设置    |
| agents.json   | ~/.claude/ 或项目/.claude/ | 子代理配置      |
| mcp.json      | ~/.claude/              | MCP 服务器配置  |
| hooks/        | ~/.claude/hooks/        | Hook 脚本    |
| skills/       | ~/.claude/skills/       | 自定义 Skills |
| rules/        | 项目/.claude/rules/       | 模块化规则      |

C. 推荐资源

官方资源

总结

Claude Code 真的很强，是一个强大的系统级 AI Agent，用好这个工具能为自己提高很多工作效率。

核心能力清单

- ✔ **Skills:** 预封装的工作流,快速复用专业能力
- ✔ **Hooks:** 事件驱动的自动化,打造个性化工作流
- ✔ **Plugins:** 完整解决方案,一键安装多功能套件
- ✔ **MCP Servers:** 外部服务集成,无限扩展能力边界
- ✔ **Subagents:** 并行处理复杂任务,提升团队协作效率
- ✔ **CLAUDE.md:** 项目记忆系统,让 AI 理解你的项目,并支持自我进化
- ✔ **Plan 模式:** 先规划后执行,减少返工提高质量(90% 时间都在用)
- ✔ **Slash Commands:** 复杂工作流一键执行,团队共享最佳实践
- ✔ **Extended Thinking:** ultrathink 深度思考模式,解决复杂问题
- ✔ **Sandbox 模式:** 安全防护机制,保护你的工作环境
- ✔ **Headless 模式:** CI/CD 集成,实现自动化工作流
- ✔ **LSP 集成:** IDE 级代码理解,实时错误检测
- ✔ **Agent SDK:** 构建自定义智能体,几十行代码创造强大工具

顶级开发者的秘诀

来自 Claude Code 创始人和 Anthropic 团队的实战经验:

1. 并行处理是效率倍增的关键

- 同时运行 5 个终端实例 + 5-10 个网页会话
- 利用系统通知和多设备协作
- 效率提升可达 1900%+

2. AI 进化机制让工具越用越聪明

- 在 PR 评论中直接 @claude 反馈
- 自动将教训写入 CLAUDE.md

- 整个团队的 AI 助手持续进化

### 3. 验证闭环是质量保证的基石

- 永远给 Claude 验证自己工作的方法
- 代码质量提升 2-3 倍
- 返工率降低到 5%

### 4. 选择合适的工具

- 简单任务: Haiku 4.5 或国产模型
- 日常开发: Sonnet 4.5
- 复杂任务: Opus 4.5 + Thinking(创始人首选)
- 追求极致效率:聪明的大模型比"快但笨"的小模型更快

### 5. 先规划后执行

- 90% 的时间使用 Plan 模式
- 你是架构师,Claude 是执行者
- 一次做对,永远比反复修改更省时间

## 设计哲学

看完这些技巧会发现 Claude Code 的设计哲学非常有意思:

- 通过 **Plan Mode**,它尊重人的决策权
- 通过 **Hooks** 和 **Sandbox**,它给人提供了控制权
- 通过 **Subagents** 和 **Automation**,它帮人分担了繁琐的执行工作
- 通过 **LSP** 和 **Agent SDK**,它提供了无限的扩展可能

*正如 Ado Kukic 所说:"用得最好的开发者,不是那些把所有事情都丢给 AI 的人,而是那些懂得何时使用计划模式、何时开启深度思考、如何设置安全边界的人。"*

Claude Code 不仅仅是一个工具,它是一个可编程、可扩展、可进化的智能开发环境。你的创造力决定了它的上限。